Lab 4 – Vibe Coding: Chat History (Claude Code)

Oct 8, 2026 · @riyana

Step 1: Read-only bug assessment

Prompt

```
Do a read-only bug assessment of this project. Do NOT edit, create, or delete
any files, and do not run git commands that change anything.

1. Read the README and summarize the project's goal and the list of features
I'm expected to add.
2. Read all the source files. Briefly describe the structure (main entry
point, key files, game loop).
3. Identify everything that is broken, which could be crashes, syntax or
import errors, logic bugs, or things that don't match the README. For each
issue give:
   - file and line number
   - what's wrong and what symptom I'd see when running it
   - severity (crash / wrong behavior / minor)
   - a one-line suggested fix (described only, not applied)
4. Tell me how to run the game, and whether anything in requirements.txt or
the setup looks off.
5. End with a numbered list of the bugs in the order you'd fix them, so I can
tackle them one at a time with a separate commit for each.

Don't fix anything yet. I'll tell you which one to do first.
```

Claude Code response

1. README: goal and tasks. The goal is a small platformer built with Pygame and object-oriented code. You debug and finish it with an AI assistant, then submit before/after gameplay videos and the chat log.

What it should already do: move with Left/Right or A/D, jump with Space/Up/W, and have gravity. Platforms catch you when you land from above. Touching the hazard or falling off the bottom ends the game. Reaching the goal at the right edge scores a point and sends you back to the start. The score is always on screen.

Features to add:

- Task 1: make landings reliable so the player never falls through a platform after a long fall.
- Task 2: a Game Over screen showing the final score that waits for input, instead of printing to the console.
- Task 3: a replay option that lets you pick a difficulty (Easy, Medium or Hard gravity/jump strength) or exit.
- Task 4: sound effects for jumping, reaching the goal, and dying.

2. Structure. `main.py` is the entry point. It sets up pygame and an 800×500 window, creates `GameEngine` at module level (line 20), and runs the loop in `main()`: clear the screen, handle events, `handle_input()`, `update()`, `render()`, flip, then tick at 60 FPS.

`game/game_engine.py` holds all the game logic:

- The level is 4 platforms, 1 hazard sitting on platform 2, and a goal line at x=740.
- `handle_event` handles jumping, which fires on key-down.
- `handle_input` handles left/right, which is checked every frame from held keys.
- `update` applies gravity, moves the player, then checks platforms, the hazard, falling off the screen, and the goal.
- `render` draws everything, plus the console print for game over.

`player.py`, `platform.py` and `hazard.py` are small data classes, each with a `rect()` method.

3. Issues

#	Location	Problem and symptom	Severity	Suggested fix
A	(setup) requirements.txt:1	Both of your Pythons are 3.14, and pygame 2.6.1 (the latest) has no 3.14 wheel. I confirmed this with a pip dry run. `pip install -r requirements.txt` will try to build from source, which usually fails on macOS. The Homebrew python3 also refuses pip install outright because of PEP 668. Symptom: you can't install, or ModuleNotFoundError: pygame.	crash (blocker)	Use a venv and either switch to pygame-ce (it has a 3.14 wheel, 2.5.8, and the same `import pygame` API) or create the venv with Python 3.12.
B	game_engine.py:55-74 with player.py:16	`on_ground` flickers when standing still. Gravity moves the player 0.6px into the ground. pygame truncates the float y to an int in Rect, so the player's rect doesn't overlap the platform that frame and `on_ground` becomes False. The next frame it overlaps and snaps back. My simulation shows it alternating True/False every frame, as long as Rect truncates floats as pygame does. Symptom: about half your jump presses do nothing. This is probably the "falling through" feel Task 1 describes.	wrong behavior	Count the player as grounded when their bottom is touching the platform top, or don't apply gravity while grounded.

#	Location	Problem and symptom	Severity	Suggested fix
C	game_engine.py:70-74	Landing only checks for overlap. Any overlap while vy >= 0 snaps the player to the platform top, even when coming from the side or below the top edge. Nothing checks whether the player's bottom crossed the platform top during the frame. Symptom: brushing a platform's side while falling pops the player up onto it. It's also the root cause the Task 1 comment describes.	wrong behavior	Use a swept check: land only if prev_bottom <= platform.y <= new_bottom and the x-ranges overlap. Also add a terminal-velocity cap on vy.
D	game_engine.py:58-67	Task 1's fall-through doesn't happen with the current numbers. Falling through a 14px platform with a 32px player needs vy > 46, which takes about 1,760px of falling. The highest possible fall here is about 180px (max vy ≈ 15). Symptom: you can't record the "before" video for the fall-through.	minor (README mismatch)	Fix C anyway. For the "before" video, show B (missed jumps) or C (side snap) instead. Don't change gravity just to show the bug.
E	game_engine.py:52-53, 104-107	On game over the game freezes silently and only prints to the terminal. The only way out is closing the window.	wrong behavior (= Task 2)	Covered by Task 2. Note that `_game_over_logged` and `engine` (created at main.py:20) will need resetting or re-creating for Task 3.
F	repo root	`game/__pycache__/*.pyc` files are committed and show as modified. There is no .gitignore. Symptom: noise in every commit.	minor	Add a .gitignore with `__pycache__/` and `*.pyc`, then `git rm --cached` the .pyc files.
G	README.md:3, README.md:26	It says "terminal-based", but it's a graphical window. "Python 3.10+" doesn't mention that 3.14 breaks plain pygame.	minor	Update the wording and give a supported Python range.

Checked and fine:

- All imports resolve, and the package layout (relative imports) works with `python main.py`.
- The level can be completed. My simulation reached the goal in about 175 frames, so there is no impossible gap.
- Hazard detection, the fall-off-the-screen check, goal scoring and the reset all behave as the README says.
- Fonts and colors are fine.

4. Running it

```
cd ~/Desktop/44_simple-platformer
python3 -m venv .venv && source .venv/bin/activate
pip install pygame-ce        # or: use python3.12 -m venv … and keep plain
pygame
python main.py
```

- requirements.txt is unpinned, and plain pygame won't install on 3.14. Change it to `pygame-ce>=2.5` or pin a Python version.
- Old .pyc files: the committed ones are cpython-312, so the starter was apparently run on 3.12.
- .gitignore: add `.venv/` to it as well.

5. Order of fixes (one commit each)

1. A: fix requirements.txt and setup so the game runs. Record your "before" video here.
2. F: add .gitignore and untrack `__pycache__`, so later commits stay clean.
3. B: fix the `on_ground` flicker so jumps register every time.
4. C + Task 1: add the swept landing check and the terminal-velocity cap.
5. G: update the README wording and Python version note (it can go with #1 if you prefer).
6. Task 2: Game Over screen.
7. Task 3: replay with difficulty, which also covers the reset issue noted in E.
8. Task 4: sounds.

Tell me which one to start with.

Step 2: New branch, fix Issue B and Issue C / Task 1

Prompt

```
Thanks. Now make changes, but first create and switch to a new branch called
lab4-changes (git switch -c lab4-changes). Don't touch main.

Scope: only changes that affect how the game plays. Do NOT edit README.md,
requirements.txt, or .gitignore, and ignore issues A, F and G from your list.
I've already fixed my Python setup separately.

Fix these two issues, one at a time, each as its own commit:

1. Issue B: the on_ground flicker, so jump presses register every frame while
standing on a platform.
2. Issues C + Task 1: replace the overlap-based landing with a swept check
(land only if the player's bottom crossed the platform top this frame and the
x-ranges overlap), and add a terminal-velocity cap on vy so landings stay
reliable after long falls.

Rules:
- Keep the changes minimal and in the existing style. Don't refactor anything
unrelated.
- After each fix, run the game logic far enough to confirm it works (e.g.
import check or a quick headless test), and tell me what you verified.
- For each commit, stage only the specific source files you changed. Don't
use "git add -A", and don't stage any __pycache__ or .pyc files.
- Use a clear commit message per fix, like "Fix on_ground flicker (Issue B)"
and "Add swept landing check and terminal velocity cap (Issue C, Task 1)".
- Stop after these two commits and show me the diff summary. Don't start Task
2, 3 or 4 yet.
```

Claude Code response

Both fixes are committed on the new `lab4-changes` branch. `main` is untouched (still at `140ee4d`), and only `game/game_engine.py` changed. I tested headlessly; I haven't played it in a window.

```
ff87f60 Add swept landing check and terminal velocity cap (Issue C, Task 1)
19fb0bd Fix on_ground flicker (Issue B)

 game/game_engine.py | 28 ++++++++++++++++++++------------
 1 file changed, 16 insertions(+), 12 deletions(-)
```

Each commit staged only `game/game_engine.py`. The `__pycache__/*.pyc` files are still modified in your working tree, as they were before.

Commit 1, Issue B (on_ground flicker). Before changing anything, I ran the real engine and confirmed the cause: pygame truncates 428.6 to 428, so `on_ground` alternated True/False every frame while standing. The fix extends the contact rect 1px below the player's feet. After the fix:

- `on_ground` stayed True on every frame across 60 frames of standing.
- Jump worked when pressed on each of 10 consecutive frames (before, it only worked on every other frame).
- Walking off a ledge still falls, and walking into the hazard still kills.

Commit 2, Issue C and Task 1 (swept landing and speed cap).

- Swept check: the player lands only if their bottom crossed or rests on the platform top this frame and the x-ranges overlap. It uses the float positions, so standing contact stays stable without the 1px workaround. Commit 2 therefore replaces commit 1's code; the flicker fix itself is kept, and I re-tested it.
- Speed cap: `MAX_FALL_SPEED = 14`, the same as a platform's thickness.
- Results:
  - Brushing the side of platform 3 below its top no longer pops the player on top (reproduced before the fix, gone after).
  - A fall from y=-5000 tops out at vy=14 and lands exactly at y=428.
  - With the cap turned off and a 200px drop in a single frame, the swept check alone still catches the platform.
  - Gaps, the hazard and goal scoring all still work.
  - A search over every input sequence still finds a way to complete the level.

Things to know:

- Test environment: neither `python` nor `python3` on this machine has pygame, and I couldn't find your setup. I tested in a throwaway venv in my scratchpad with `pygame-ce` 2.5.8, outside the project, using `PYTHONDONTWRITEBYTECODE` so no `.pyc` files were rewritten.
- Level feel: the old side-snap actually made the level slightly easier. The fastest path the search found went from 175 to 188 frames, so jumping onto the raised middle platform now needs a cleaner jump.

- Speed cap in normal play: it rarely kicks in, because the longest fall in this level only reaches about 15 px/frame.

I stopped there and haven't started Tasks 2–4.

Step 3: The hazard on platform 2 is too wide to clear

Prompt (diagnose first)

```
Before Task 2, one gameplay issue: I can't get past the red platform (the one
with the hazard, around platform 2). The level has become too hard after the
landing fix.

Step 1, diagnose only (no edits yet):
- Compare the original behavior on main (use "git show
main:game/game_engine.py" and run it in your headless test) with the current
lab4-changes behavior. Is this a regression from my two commits, or was it
always this hard?
- Work out the numbers: max jump height, horizontal distance covered in a
jump, the platform gaps, and the hazard's size and position. Tell me exactly
why a normal player can't clear that section (e.g. the hazard is too wide,
the gap is too long, or the jump can't reach).

Step 2, propose the smallest fix and wait for my OK:
- Prefer making the level more forgiving (e.g. shrink the hazard, nudge a
platform or the hazard's position) over changing gravity or jump strength,
because Task 3 will use gravity and jump strength for difficulty levels.
- It must remain beatable by a normal player with a reasonable jump, not just
barely possible by a frame-perfect search.
- Don't reintroduce the fall-through or side-snap bugs.

Don't edit the README, requirements.txt or .gitignore. Don't commit anything
until I've approved the plan.
```

Claude Code response (diagnosis and proposed fix)

Step 1: Diagnosis. Main vs. lab4-changes. I ran the same headless tests against `git show main:game/game_engine.py` and the branch:

Test	main	lab4-changes
Hold Right the whole level, time your jumps: number of first-jump timings that reach the goal	0	0
Jump from platform 1 onto the safe strip before the hazard (frame window)	4	5
Get from that strip to the strip after the hazard (stop, jump, let go of Right)	20 of 210 attempts	20 of 210 attempts
Jump from the strip after the hazard to platform 3 (frame window)	3	5

The branch is very slightly easier, because the flicker fix no longer drops jump presses. I was wrong in my first assessment to say the level "can be completed": that was based on a search that tries every input combination frame by frame, not on what a person can do.

The numbers:

- Jump: peaks at 114px, stays in the air about 40 frames, and covers about 156–160px if you hold Right.
- Platforms: P1 runs from x=0 to 160 (y=460). Then a 60px gap. P2 runs from 220 to 360 (y=460). Then a 60px gap, and P3 runs from 420 to 540, raised 60px (y=400). Then a 60px gap and a 60px drop to P4, from 600 to 780.
- Hazard: x=240 to 340 and 14px tall, so it covers 100 of P2's 140px. That leaves a 20px safe strip on each side, but the player is 24px wide, so each strip has only 5 standing positions.

Why a normal player can't get past it:

- You can't jump over the hazard from P1. Clearing it from P1's edge means covering at least 184px, and a jump only covers about 156px.
- Landing on the left strip needs a 5-frame (83ms) jump timing.
- From the left strip, you're stuck. A running jump overshoots the right strip and drops into the next gap.
- P3 is out of reach: you'd need to take off from about x≥268, which is inside the hazard.
- The only way on is to jump and let go of Right at an exact frame to land on the right strip, which works about 10% of the time. From the right strip to P3 is another 5-frame window.

So you'd need three precise inputs in a row. The hazard is simply too wide for this jump.

Step 2: Proposed fix (waiting for your OK). I'd change one number, the hazard width from 100 to 30 at `game_engine.py:35`:

```
self.hazards = [Hazard(240, ground_y - 14, 30)]
```

The hazard stays where it is, at the left edge of P2, but now spans 240–270. The new route is: jump the gap and the hazard together, land on P2, then jump up to P3.

With the change (lab4-changes)	Before	After
Hold Right, time the first jump	0 frames	10 frames (~167ms)
Second jump (P2 to P3) after a working first jump	n/a	11–20 frames
Stop near P1's edge, jump, hold Right	almost never works	works from anywhere in the last 40px of P1

Why this option:

- Gravity and jump strength are untouched, which leaves them free for Task 3.
- The collision code is untouched, so the swept landing check and the speed cap stay as they are and neither old bug can come back. The hazard is 30px wide, still wider than the player, so it remains a real obstacle: jump too early and you land on it.
- I tested other placements: 30px at x=250 and 40px at x=240 give 8-frame first-jump windows instead of 10. 30px at x=260 gives only 5 frames. 30px at x=320 (far right of P2) makes the first jump easy, but the second jump's window drops to as little as 1 frame.

If you approve, I'll make that one-line change, rerun the whole-level test and the collision checks from your last two commits, and commit only `game/game_engine.py` as something like "Shrink hazard so platform 2 is clearable".

Prompt (approval)

```
Approved: change the hazard width from 100 to 30 at game_engine.py:35,
keeping it at x=240 on platform 2.

Before committing:
- Rerun the whole-level test and the collision checks from my last two
commits.
- Also check the rest of the route by timing windows rather than brute-force
search: P3 to P4 (the 60px gap and 60px drop), and P4 to the goal at x=740.
Tell me the frame windows for each jump, so I know the whole level is
playable by a human, not just the first section.
- If any other jump has a window under ~5 frames, tell me before committing
instead of changing it silently.

Then make one commit, staging only game/game_engine.py. Message: "Shrink
hazard so platform 2 is clearable". Stop after the commit and show me the
diff summary. Don't start Task 2.
```

Claude Code response (commit)

The hazard is now 30px wide, still at x=240, and that's committed on lab4-changes as `5674e5d`. Every test passed, and no jump on the route has a timing window under 5 frames, so there was nothing to raise before committing. Only `game/game_engine.py` was staged; the .pyc files are still unstaged.

```
5674e5d Shrink hazard so platform 2 is clearable
ff87f60 Add swept landing check and terminal velocity cap (Issue C, Task 1)
19fb0bd Fix on_ground flicker (Issue B)

This commit:    game/game_engine.py | 2 +-   (Hazard width 100 → 30)
Branch vs main: game/game_engine.py | 30 ++++++++++++++++++++++++++++++++++
```

Timing windows across the level, assuming you hold Right and time your jumps (60 frames = 1 second):

Section	Jump needed?	Window
P1 → P2 (over the gap and hazard)	yes	10 frames (~167ms) from a run, or a standing jump anywhere in the last 40px of P1
P2 → P3 (60px up)	yes	11–20 frames, depending on where you landed on P2

Section	Jump needed?	Window
P3 → P4 (60px gap, 60px drop)	no: just walk off the edge	Pressing jump is also always safe: 50–69 of 50–69 frames work, depending on where you landed on P3
P4 → goal at x=740	no: just walk	Jumping is also safe on every frame (7–38 frames of P4 left to run, depending on landing spot)

For the last two rows, I tested every P3 and P4 landing spot that the working routes actually produce. Neither section had a single failing frame.

Rerun checks:

- Issue B: the player stays grounded on every frame while standing, and jump registered 10 times out of 10.
- Issue C: brushing the side of P3 below its top no longer pops the player on top.
- Task 1: falls from y=-5000 top out at vy=14. They land correctly on P4 and on P2 right of the hazard. A fall onto the hazard at x=250 still kills. With the speed cap off, a 200px single-frame fall still lands.
- Unchanged behavior: the gap still kills, walking into the hazard still kills, and the goal still scores and resets you.

I haven't started Task 2.

Task 2: Game Over screen

Prompt

```
Good work. Staying on the lab4-changes branch, now implement Task 2 from the
README: a Game Over screen.

Requirements:
- When the game ends (hazard or falling off the bottom), show a Game Over
screen in the pygame window with the text "GAME OVER" and the final score.
- It must wait for player input instead of printing to the console. Remove
the console print and the _game_over_logged logic.
- Pressing Enter or Esc, or closing the window, exits the game cleanly
(pygame.quit and a normal exit). Ignore key presses for the first ~0.5
seconds so a jump key pressed at the moment of death doesn't dismiss the
screen instantly.
- The window close button must work while the screen is showing.
- Design it so Task 3 (replay with a difficulty menu) can replace the "exit"
step easily. Use a simple state on the engine (e.g. "playing" / "game_over")
rather than a one-off hack, but don't build any replay or difficulty features
yet.

Rules:
- Only change code that affects the game. Don't edit the README,
requirements.txt or .gitignore.
- Keep the changes minimal and in the existing style.
- Check that it works as far as you can headlessly, and tell me what you
verified and what you couldn't.
- Make one commit, staging only the source files you changed (no git add -A,
no __pycache__/.pyc). Message: "Add Game Over screen (Task 2)".
- Stop after the commit and show me the diff summary. Don't start Task 3 or
4.
```

Claude Code response

The Game Over screen is in and committed on `lab4-changes` as `6bbe7f0`, staging only `game/game_engine.py` and `main.py`. I tested it headlessly, including the real `main()` loop, but I couldn't test a real window or keyboard on your Mac.

```
6bbe7f0 Add Game Over screen (Task 2)
5674e5d Shrink hazard so platform 2 is clearable
ff87f60 Add swept landing check and terminal velocity cap (Issue C, Task 1)
19fb0bd Fix on_ground flicker (Issue B)

This commit:     game/game_engine.py | 55 ++++++---   main.py | 2 ++   (47+,
10-)
Branch vs main:  game/game_engine.py | 85 ++++++++---  main.py | 2 ++
(64+, 23-)
```

What changed

- `game_engine.py`:
  - The `game_over` flag is replaced by `self.state`, which goes "playing" → "game_over" → "exit".
  - Both kinds of death (hazard and falling off the bottom) now call a new `end_game()`, which also records the time of death.
  - `handle_event` handles keys by state. Playing: jump keys work as before. Game Over: Enter, keypad Enter or Esc set "exit", but only after `GAME_OVER_INPUT_DELAY_MS = 500`.
  - The console print and `_game_over_logged` are gone. `render_game_over()` dims the frozen level and draws "GAME OVER", "Final score: N" and "Press Enter or Esc to exit".
- `main.py`: two lines that end the loop when `engine.state == "exit"`. The existing quit handling covers the close button and then calls `pygame.quit()`.
- Hook for Task 3: the replay menu goes in the `game_over` branch of `handle_event`, in place of the line `self.state = "exit"`.

Verified headlessly

- Engine:
  - Both kinds of death set the state to `game_over`, and the level stays frozen afterwards.
  - Jump keys do nothing on the Game Over screen.
  - Enter and Esc are ignored within the first 500ms; other keys are ignored even after it.
  - After 500ms, Enter, keypad Enter and Esc each set the state to `exit`.
  - Reaching the goal while playing still scores.

    - Nothing is printed to the console.
- Rendering: I drew one frame to an image and checked it. The text is centred and readable over the dimmed level, and the final score appears.
- The real `main()` loop, forced to die on the first frame, with timed events injected:
    - Enter at 200ms was ignored, and Enter at 900ms exited.
    - Esc at 700ms exited.
    - Closing the window at 250ms (inside the delay) and at 1200ms both exited.
    - Space and Up presses had no effect, and the screen stayed up until the window was closed at 2s.
    - Every run returned normally with exit code 0, and pygame was shut down afterwards.

Couldn't verify (worth a quick manual check)

- The macOS window close button. I sent pygame's quit event, which is what that button produces, but didn't click the button itself.
- Real keyboard input and how the screen looks with the fonts on your system.
- A full run in your own Python setup.

I haven't started Task 3 or 4.

Task 3: Replay with difficulty selection

Prompt

```
Good work. Staying on lab4-changes, now implement Task 3 from the README: a
replay option with a difficulty choice.

Behavior:
- On the Game Over screen, show a menu under the final score: Easy, Medium,
Hard, Exit. The player picks with Up/Down plus Enter. The number keys 1/2/3
select Easy/Medium/Hard directly. Esc or the Exit option exits cleanly, and
the window close button must still work. Keep the existing 500ms input delay.
- Choosing a difficulty restarts the level: the player goes back to the
start, velocity and on_ground are reset, the score resets to 0, and the state
returns to "playing". Nothing from the previous run may leak into the new one
(no stuck state, no leftover timers).
- Difficulty changes gravity and jump strength. A fresh game starts on
Medium, and Medium must use exactly the current gravity and jump values so
the level stays as tuned.
```

```
  - Easy: more forgiving (e.g. slightly lower gravity and/or stronger jump).
  - Hard: less forgiving (e.g. slightly higher gravity and/or weaker jump).
  - Keep the differences noticeable but modest. All three must stay beatable
by a human.
- The replay can be repeated as many times as the player wants, and each
death shows the menu again.

Implementation notes:
- Put the difficulty presets in one clear place (e.g. a small dict of name ->
gravity, jump strength) rather than scattering numbers.
- Add a reset/restart method on the engine instead of rebuilding objects in
main.py. If main.py needs a small change, keep it minimal.
- The swept landing check, MAX_FALL_SPEED cap, hazard size and level layout
must stay as they are. Don't reintroduce the old bugs.

Verification (headless is fine, tell me what you checked and what you
couldn't):
- For each of Easy, Medium and Hard, measure the timing windows for every
jump on the route (P1 to P2, P2 to P3, P3 to P4 and the walk to the goal), as
you did before. Tell me the max jump height, jump distance and window for
each difficulty. If any required jump on any difficulty has a window under ~5
frames, adjust the preset values and re-measure before committing.
- Replay twice in a row, then exit, in the real main() loop. Confirm the
score and state reset each time.
- Selecting by arrow keys, by number keys, and by Esc/Exit all work. Keys
pressed within the first 500ms are ignored.

Rules:
- Only change code that affects the game. Don't edit the README,
requirements.txt or .gitignore.
- Make one commit, staging only the source files you changed (no git add -A,
no __pycache__/.pyc). Message: "Add replay with difficulty selection (Task
3)".
- Stop after the commit and show me the diff summary. Don't start Task 4.
```

Claude Code response

Task 3 is committed on lab4-changes as `c0b234d`. Only `game/game_engine.py` changed; `main.py` didn't need an edit, because a replay never reaches the "exit" state. I verified everything headlessly in the real engine and `main()` loop; I couldn't test a real window or keyboard on your Mac.

```
c0b234d Add replay with difficulty selection (Task 3)
6bbe7f0 Add Game Over screen (Task 2)
5674e5d Shrink hazard so platform 2 is clearable
ff87f60 Add swept landing check and terminal velocity cap (Issue C, Task 1)
19fb0bd Fix on_ground flicker (Issue B)

This commit:     game/game_engine.py | 75 +++++++++--   (61+, 14-)
Branch vs main:  game/game_engine.py | 140 ++++++++---  main.py | 2 +   (115+,
27-)
```

What changed

- Presets: a single `DIFFICULTIES` dict maps each name to (gravity, jump strength). Easy: 0.55, -12. Medium: 0.6, -12 (exactly the original values). Hard: 0.65, -12. A fresh game starts on Medium.
- `restart(difficulty)`: creates a new Player at the start position, so velocity and `on_ground` can't carry over. Resets the score, state, Game Over timer and highlighted menu entry, and applies the preset. `__init__` uses the same method, so a new game and a replay start the same way.
- Menu: Easy, Medium, Hard and Exit appear under the final score, with the current difficulty highlighted. Up/Down move the highlight and wrap around; Enter selects. 1/2/3 (keypad too) pick a difficulty directly. Esc exits.
- Unchanged: the swept landing check, `MAX_FALL_SPEED`, the hazard and the level layout. I confirmed this with a diff.

Physics and timing windows per difficulty (holding Right, 60 frames = 1s)

	Easy	Medium	Hard
Max jump height	125px	114px	105px
Airtime / reach	43f / 172px	40f / 160px	36f / 144px
P1 → P2 (over gap and hazard)	14 frames	10 frames	7 frames
P2 → P3	8–21 frames	11–20 frames	15–20 frames
P3 → P4	walk off, no jump needed	same	same

	Easy	Medium	Hard
P4 → goal	walk, no jump needed; jumping is never fatal	same	same

Two things to know about the presets:

- Why the difference is only about ±8% gravity: I also tried stronger spreads (Easy 0.5, Hard 0.7, and weaker or stronger jumps), but they pushed some required jumps down to 4–6 frames. A floatier Easy jump overshoots the 140px platform 2, and a shorter Hard jump barely clears the gap, so I kept the moderate values.
- Hard has a few optional failing jumps between P3 and P4: 10 of about 1,250 tested jump timings fail. If you jump the moment you land at P3's left edge, Hard's shorter reach falls into the gap. Simply walking off always works.

Verified in the real main() loop

- Two replays, then Exit: Down + Enter chose Hard, then 1 chose Easy, then Up (wrapping round to Exit) + Enter quit.
- Each restart logged the expected state: start position, vx=0, vy=0, on_ground=False, score 0, the right gravity and jump values, and the timer reset. The next death correctly showed that run's own score.
- Other ways in and out: 2 then Esc, keypad 3 then closing the window during the input delay, and closing the window mid-play all exited with code 0, and pygame was shut down each time.
- 500ms input delay: keys pressed inside it after each death (3, 1 and Esc) were all ignored.

Engine-level checks

- Space and W are ignored on the menu.
- Jumping works again after a replay.
- The flicker fix, the side-snap fix and the speed cap all still hold on all three difficulties.
- Rendering: I checked a screenshot of the menu, and added a little spacing above the hint line after seeing it.

Correction to my Task 2 report: that main-loop test scheduled keys with `pygame.time.set_timer`, which keeps only one timer per event type. So "Enter at 200ms is ignored" was probably never really sent in that run; only the engine-level test covered it. This time I injected events on a time schedule, and the delay is confirmed in the real loop.

Couldn't verify: real keyboard input, clicking the macOS close button, how it looks with your system fonts, and whether the difficulties feel noticeably different in actual play.

I haven't started Task 4.

Task 4: Sound effects

Prompt

```
Good work. Staying on lab4-changes, now implement Task 4 from the README:
sound effects for jumping, reaching the goal, and dying.

Requirements:
- Three distinct sounds: jump, goal reached, and death. Each should be short
(roughly 0.1–0.6s) and clearly different, e.g. a quick rising blip for the
jump, a bright two- or three-note rising chime for the goal, and a descending
tone for death.
- There are no audio files in the repo, so generate the sounds in code using
only the Python standard library plus pygame (e.g. build the samples with the
array module or struct and create them with pygame.mixer.Sound(buffer=...)).
Do NOT add numpy or any new dependency, and do not edit requirements.txt.
Don't add audio files either.
- Play each sound exactly once per event:
  - Jump: only when a jump actually happens (the player is grounded and the
press registers), not on ignored key presses and not on the Game Over/menu
screens.
  - Goal: when the player scores at the goal.
  - Death: once at the moment of dying (hazard or falling off the bottom),
not every frame while the Game Over screen is up.
- Keep the volume moderate so it isn't harsh.
- Be robust: if the audio device or mixer can't initialize (no sound card,
headless machine), the game must still run normally with no sound and no
crash. Make sure the mixer is initialized before the sounds are created, and
generate the sounds once at startup, not on every event.
- Replays (Task 3) must keep working: sounds play correctly across multiple
restarts and don't pile up or get cut off weirdly.
- Keep everything else unchanged: the swept landing check, MAX_FALL_SPEED,
hazard, difficulty presets, Game Over screen and menu.

Rules:
- Only change code that affects the game. Don't edit the README,
requirements.txt or .gitignore.
```

```
- Keep the changes minimal and in the existing style. A small helper for
creating the sounds is fine.
- Verify headlessly using SDL_AUDIODRIVER=dummy: sounds are created, each
event triggers its sound exactly once (e.g. by checking or mocking the play
calls), jumps while airborne play nothing, death plays once not per frame,
and the game still runs when the mixer init is forced to fail. Tell me what
you checked and what you couldn't (I will listen to them myself).
- Make one commit, staging only the source files you changed (no git add -A,
no __pycache__/.pyc). Message: "Add sound effects for jump, goal and death
(Task 4)".
- Stop after the commit and show me the diff summary.
```

Claude Code response

Task 4 is committed on `lab4-changes` as `286dbdb`. Each event plays its sound exactly once, and the game runs silently if audio isn't available. I verified this headlessly with the dummy audio driver; I couldn't hear the sounds, so how they actually sound is for you to judge.

```
286dbdb Add sound effects for jump, goal and death (Task 4)
c0b234d Add replay with difficulty selection (Task 3)
6bbe7f0 Add Game Over screen (Task 2)
5674e5d Shrink hazard so platform 2 is clearable
ff87f60 Add swept landing check and terminal velocity cap (Issue C, Task 1)
19fb0bd Fix on_ground flicker (Issue B)

This commit:  game/sounds.py 58 (new) | game/game_engine.py 13 |
game/player.py 3   (73+, 1-)
Branch vs main: 4 files changed, 187 insertions(+), 27 deletions(-)
```

Only those three files were staged: no `.pyc` files and no audio files. `requirements.txt` is unchanged.

What changed

- `game/sounds.py` (new): `load_sounds()` generates the three sounds once at startup, using only `array` and `math`.
  - Jump: a 0.12s blip rising from 380 to 760Hz.
  - Goal: a 0.45s C–E–G chime.
  - Death: a 0.5s tone falling from 440 to 110Hz.
  - Volume and clicks: all three peak at 25% of full volume, with 5ms fade-in/out to avoid clicks.

    - Mixer setup: samples are built in whatever format the mixer actually reports, and the mixer is initialized first if it isn't already.
    - No audio: it returns `{}`, so the game runs silently.
- `player.py`: `jump()` now returns whether a jump actually happened.
- `game_engine.py`: sounds load once in `__init__` (not in `restart()`), and a `play_sound()` helper plays them. Jump: plays only when `jump()` returns True. Goal: plays where the score goes up. Death: plays in `end_game()`, which runs once per death.
- Unchanged: collision, the speed cap, the hazard, the presets and the menu. I confirmed this with a diff.

Verified headlessly (`SDL_AUDIODRIVER=dummy`)

- Sounds are created correctly. I checked lengths (0.12s / 0.45s / 0.50s) and peak level (0.25) in five mixer formats: 44.1kHz stereo 16-bit (the default here), 22kHz mono, 32-bit float, 8-bit, and 48kHz. Generation takes under 0.03s. The pitch measured from the samples goes up for the jump, steps through ~525→662→787Hz for the goal, and goes down for death.
- Exactly once per event, checked with counters wrapped around each sound:
    - A grounded jump plays once. 13 presses while airborne played nothing.
    - Each goal plays once.
    - A death followed by 300 more frames and menu key presses still played the death sound only once.
    - Three replays each added exactly one death sound.
    - The sounds were generated once and never regenerated on restart.
- Randomized check: 20,000 frames of random keys with replays matched exactly: 209 jump sounds for 209 real jumps, and 149 death sounds for 149 deaths. Random play never reached the goal, so goals rely on the test above.
- Real playback path: `play_sound()` makes the mixer busy for all three sounds. The real `main()` loop with two replays and Esc exited with code 0.
- No audio available, tested three ways: a nonexistent audio driver in the real `main()` loop (replayed twice, then exited); `mixer.init` forced to raise `pygame.error`; the mixer module missing entirely. In each case `sounds = {}`, and jumping, scoring, dying and replaying all worked without errors.

Fixed along the way: pygame reports its 32-bit float format as `-32`. Before I added that, a float-format mixer would silently have produced no sound.

Couldn't verify (for your listening test)

- Whether the sounds are pleasant and clearly distinct, and whether 25% volume feels right on your speakers.
- Real-time overlap on actual hardware. The mixer has 8 channels and each sound is at most 0.5s, which comes out shorter than the 500ms menu delay plus a new jump, so nothing should pile up or cut off. I only checked this by reasoning, not by listening.

All four README tasks are done on `lab4-changes`. Nothing has been merged into `main`.

Final commit history on lab4-changes

Commit	What it does
`19fb0bd`	Fix on_ground flicker (Issue B)
`ff87f60`	Add swept landing check and terminal velocity cap (Issue C, Task 1)
`5674e5d`	Shrink hazard so platform 2 is clearable
`6bbe7f0`	Add Game Over screen (Task 2)
`c0b234d`	Add replay with difficulty selection (Task 3)
`286dbdb`	Add sound effects for jump, goal and death (Task 4)